

Session 1 · Deck 1.B2

Dynamic Programming: Why It Works and How It Computes

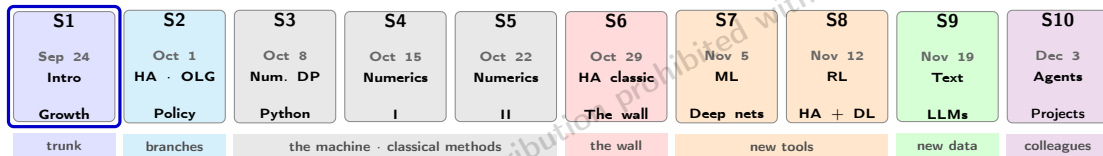
Zhigang Feng

Thursday, September 24, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

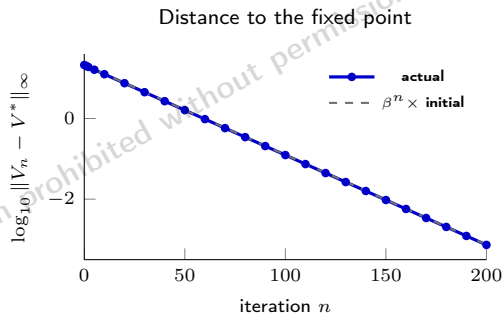
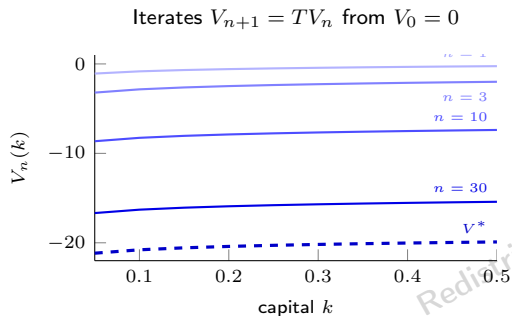
▼ you are here: 1.B2



Build it on paper → solve it on the machine → push it to the frontier with AI

This deck: the theory that makes the Bellman equation trustworthy — and the four design choices that turn it into an algorithm.

Watch Value Function Iteration Converge — Exactly



Saving rate implied by V_n : $n=1$: 0.255 · $n=2$: 0.315 · $n=3$: 0.333 · $n=4$: 0.339 · $n=8$: 0.342 = $\alpha\beta$.

The distance shrinks by exactly β per iteration (~ 270 iterations for 10^{-6}), yet the **policy is right after 8**: the reason theory guarantees convergence, and the reason smarter algorithms exist.

Brock–Mirman: $u = \ln c$, $y = k^{0.36}$, $\delta = 1$, $\beta = 0.95$; $V_n = a_n + b_n \ln k$ computed exactly; sup norm over $k \in [0.05, 0.5]$.

Sequence Problem $\Leftrightarrow$ Bellman Equation

- **Principle of optimality** (Bellman 1957):
the tail of an optimal plan is optimal from the state it reaches.
- So “choose an infinite sequence” becomes “choose today, then face the *same* problem tomorrow”:

$$v(k) = \max_{k' \in \Gamma(k)} \{F(k, k') + \beta v(k')\}.$$

- Assumptions: $\Gamma(k)$ nonempty; lifetime utility well defined along every feasible path (automatic if F is bounded).

	Sequence $\Rightarrow$ Bellman	Bellman $\Rightarrow$ Sequence
Values	The sequence value v^{SP} solves the Bellman equation.	A solution v with $\beta^t v(k_t) \rightarrow 0$ on feasible paths is v^{SP} .
Plans	An optimal plan satisfies the Bellman equation period by period.	A plan generated by the Bellman policy (with a limit condition) is optimal.

Stokey, Lucas & Prescott (1989), Theorems 4.2–4.5; stochastic analogues in Ch. 9.

The limit conditions play the role of **transversality**: they rule out value “escaping to infinity.” With them, solving the Bellman equation and acting on its policy gives **the same values and the same paths** as the sequence problem.

Existence, Uniqueness, Convergence: The Contraction Mapping Theorem

The **Bellman operator** on bounded continuous functions (sup norm), with $s = (k, z)$ and $c = zf(k) + (1 - \delta)k - k'$:

$$(Tv)(s) = \max_{k'} \{u(c) + \beta \mathbb{E}[v(s') \mid s]\}.$$

Theorem (Banach). If $\|Tv - Tw\| \leq \beta\|v - w\|$ on a complete space, T has a **unique** fixed point v^* , and $v_n = T^n v_0 \rightarrow v^*$ from **any** v_0 :

$$\|v_n - v^*\| \leq \frac{\beta^n}{1 - \beta} \|v_1 - v_0\|,$$

$$\|v_{n+1} - v^*\| \leq \frac{\beta}{1 - \beta} \|v_{n+1} - v_n\|.$$

Blackwell's sufficient conditions

If T is (i) *monotone*: $v \leq w \Rightarrow Tv \leq Tw$, and (ii) *discounting*: $T(v+a) \leq Tv + \beta a$ for constants $a \geq 0$, then T is a contraction.

Proof in two lines. $v \leq w + \|v - w\|$, so $Tv \leq T(w + \|v - w\|) \leq Tw + \beta\|v - w\|$. Swap v, w : $\|Tv - Tw\| \leq \beta\|v - w\|$.

The second bound is the **stopping rule** — the standard *a posteriori* bound of the fixed-point theorem: if successive iterates differ by ε , you are within $\beta\varepsilon/(1 - \beta)$ of the truth — **19 times** ε when $\beta = 0.95$.

Properties of the Solution — and What Algorithms Do with Them

Property	Why it holds	Computational payoff
v continuous; policy upper hemicontinuous	Theorem of the maximum (Berge)	Grids and interpolation are legitimate approximations
v strictly increasing and strictly concave; policy g a continuous function	u, f concave; convex feasible set	Unique maximizer: golden-section or binary search on k' (S4)
$g(k)$ increasing in k	concavity of the problem	Monotone search: start where the previous grid point stopped (S4)
v differentiable: $v'(k) = u'(c)[f'(k) + 1 - \delta]$	Benveniste–Scheinkman (interior solution)	Envelope + first-order condition $\Rightarrow$ Euler equation $\Rightarrow$ time iteration, perturbation, Euler errors (S3–S5)

Euler equation from the Bellman equation. FOC: $u'(c) = \beta \mathbb{E} v'(k', z')$. Envelope:

$v'(k, z) = u'(c)[zf'(k) + 1 - \delta]$. Substitute one period ahead:

$$u'(c) = \beta \mathbb{E} \left[u'(c')(z' f'(k') + 1 - \delta) \right].$$

The Fine Print

1. Unbounded utility

$\ln c$ and CRRA with $\sigma \geq 1$ are unbounded, so the sup-norm argument fails globally. Fixes: a compact state space $[k_{\min}, k_{\max}]$ (the growth model has one) or weighted norms (Boyd 1990).

2. Non-convexities

Discrete choices, fixed costs, default: v need not be concave, policies can jump, and Euler-equation methods can fail. Remedies include taste shocks and lotteries.

3. Equilibrium is not a contraction

The contraction solves the *household's* problem *given* prices and laws of motion. Whether an equilibrium exists, is unique, or is continuous is a **separate fixed point**. Monotone-operator arguments help in distorted economies (Coleman 1991; Greenwood–Huffman 1995); they can also fail (Santos 2002).

4. Convergence $\neq$ accuracy

An iteration that converged on a coarse grid still gives a wrong answer. Error bounds for approximate DP (Santos & Vigo-Aguiar 1998) and Euler-equation residuals (Santos 2000) measure what is left.

*Point 3 is the gateway to heterogeneous agents: from S2 on, the hard part is the **equilibrium** fixed point, not the Bellman equation.*

From Theory to Algorithm: Four Design Choices

1. Which equation?

Bellman (values)
Euler (first-order)
equilibrium residuals
S1, S3

2. How to represent functions?

discrete grid
grid + interpolation
polynomials (projection)
local Taylor (perturbation)
neural networks
S3–S5, S7

3. How to take expectations?

Markov-chain sums
Gaussian quadrature
Monte Carlo sampling
S4, S8

4. How to solve?

fixed-point iteration
policy iteration
root-finding for coefficients
gradient descent on residuals
S3–S5, S7–S8

Every method in this course — VFI, perturbation, projection, deep equilibrium nets, reinforcement learning — is **one row picked from each column**. Learn the columns, and new methods stop looking like magic.

Value Function Iteration on a Grid

Algorithm: VFI

1. Grids $\{k_i\}_{i=1}^{N_k}$, $\{z_j\}_{j=1}^{N_z}$ with transition matrix Π ; guess v_0 .
2. For every (k_i, z_j) , with $c = z_j f(k_i) + (1 - \delta)k_i - k'$:
$$v_{n+1}(k_i, z_j) = \max_{k' \in \{k_l\}} \left\{ u(c) + \beta \sum_m \Pi_{jm} v_n(k', z_m) \right\}$$
3. Stop when $\|v_{n+1} - v_n\| < \epsilon$; record the policy $g(k_i, z_j)$.

What it costs

- Per iteration: $N_k \times N_z$ points $\times N_k$ choices $\times N_z$ expectation terms.
- Iterations $\approx \ln \epsilon / \ln \beta$: **270** for $\beta = 0.95$, **1,375** for $\beta = 0.99$ (quarterly), at $\epsilon = 10^{-6}$.
- The policy settles long before the values (hook slide).

S3 codes this in 25 lines; **S4** makes it 100× faster with monotonicity, concavity and Howard steps.

Policy Iteration and the Policy Improvement Theorem

Evaluate a fixed policy g exactly — a linear system:

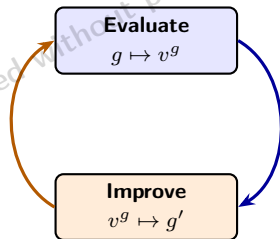
$$v^g = R_g + \beta P_g v^g \Rightarrow v^g = (I - \beta P_g)^{-1} R_g.$$

Improve greedily:

$$g'(s) \in \arg \max_a \left\{ R(s, a) + \beta \sum_{s'} P(s' | s, a) v^g(s') \right\}.$$

Theorem (Howard 1960). $v^{g'} \geq v^g$ everywhere; on a finite problem the loop reaches the optimum in finitely many steps.

Why: $v^g \leq T^{g'} v^g$; apply the monotone contraction $T^{g'}$ repeatedly: $v^g \leq T^{g'} v^g \leq (T^{g'})^2 v^g \leq \dots \rightarrow v^{g'}$.



Modified policy iteration: m evaluation steps per improvement — $m = 1$ is VFI, $m = \infty$ is policy iteration. This *evaluate–improve* loop is the backbone of reinforcement learning (1.B3).

Beyond VFI: Euler-Equation Methods and the Rest of the Toolkit

Method	Idea	Representation	Scope	Where
Time iteration	Given c_n , solve $u'(c) = \beta \mathbb{E}[u'(c_n(k', z'))R']$ point by point; a monotone operator that also works in distorted economies	grid + interpolation	global	S3
Perturbation	Taylor-expand the policy around the steady state	local polynomial	local	S5
Projection	Choose coefficients so residuals vanish at nodes (collocation) or on average (Galerkin)	global polynomials	global	S5
Neural networks	Minimize mean squared residuals over sampled states	neural network	global, high-dim	S7–S8

Value-based methods need only the Bellman equation; Euler-based methods need **differentiability** — exactly the properties on the previous slides.

How Do You Know the Answer Is Right?

Euler equation errors (Judd 1992): at points *off* the solution grid, compute the consumption the Euler equation *asks for*,

$$\tilde{c}(k, z) = (u')^{-1} \left(\beta \mathbb{E} [u'(c(k', z')) (z' f'(k') + 1 - \delta)] \right),$$

and compare it with the computed policy:

$\mathcal{E}(k, z) = |1 - \tilde{c}(k, z)/c(k, z)|$, a **unit-free** consumption mistake: $\log_{10} \mathcal{E} = -3$ means \$1 per \$1,000 of consumption. Policy and value errors are of the same order as these residuals (Santos 2000).

The verification checklist

1. A **closed form** when one exists (Brock–Mirman).
2. **Euler errors** on an independent set of points.
3. **Refinement**: does the answer move when the grid or order changes?
4. **Cross-method agreement** (Aruoba, Fernández-Villaverde & Rubio-Ramírez 2006).
5. **Simulation tests** for aggregate laws of motion (Den Haan & Marcet; S6).

Course rule: no number without a benchmark and an accuracy check — including numbers your AI assistant produced.

The Curse of Dimensionality, Counted

Source 1: the grid. m points per state variable, d states:

d	points ($m = 100$)	one value table
2	10^4	0.08 MB
4	10^8	0.8 GB
6	10^{12}	8 TB

Source 2: the expectation. With 7 nodes per shock:

- 3 shocks: $7^3 = 343$ terms at every grid point, every iteration;
- following one state forward 20 periods:
 $7^{20} \approx 8 \times 10^{16}$ shock histories.

A distribution as a state (heterogeneous agents with aggregate risk) makes d effectively infinite.

Classical responses: sparse grids, projection, parallel computing (S4–S6). **The modern response** attacks both sources at once: *sample* the expectation and *learn* the function — which is where reinforcement learning enters (1.B3).

Takeaways and Bridge

1. The Bellman equation gives the **same values and paths** as the sequence problem, under limit conditions that play the role of transversality.
2. Blackwell $\Rightarrow$ contraction $\Rightarrow$ a **unique** solution, found by iteration from **any** guess, with computable error bounds.
3. Concavity, monotonicity and differentiability are not decoration: **algorithms exploit them**.
4. The theory solves the individual problem; **equilibrium is a separate fixed point**.
5. Every method is a choice of equation, representation, expectation, and solver; **verification** is not optional.

Next (1.B3): the evaluate-improve loop you just saw is also how an AlphaGo-style agent learns. What changes when we *sample* instead of *integrate* — and what changes again when the agent is one of millions whose choices set the prices?